Demonstrations to Weights: A Hypernetwork Start for Test-Time Training on ARC-AGI-2

A 135M-parameter hypernetwork reads a task's demonstrations and emits a rank-16 adapter; merged into the base model, it ties plain test-time training task for task and leads it early in training.

Draft 1, 4 November 2026. ARC Prize 2026 Paper Track. 1,488 words (limit 1,500, captions and table included; title, subtitle and references excluded). Red brackets mark placeholders.

Abstract

We introduce H, a 135M-parameter hypernetwork that maps an ARC-AGI-2 task's demonstration pairs to a rank-16 LoRA adapter for NVARC's 4B grid model in one forward pass. Test-time training (TTT) drives the strongest ARC-AGI-2 systems, yet starts every task from the same weights. H encodes demonstrations with the frozen solver itself and is meta-trained on BARC and ReARC episodes, ending with first-order MAML. On 40 public evaluation tasks, H alone solves 5.0 tasks against the base's 4.0, within noise; merged into the base before TTT, it ties plain TTT on every task (10.5 each) with lower training loss through step 45 of 128. The linked submission scores **[PLACEHOLDER: v19 public and private scores]**.

1 Introduction

ARC-AGI-2 [1] gives each task a few demonstration pairs of colored grids and asks for the outputs of new test inputs. Tasks are new, so winning systems adapt weights per task: they fine-tune a LoRA adapter [2] on augmented demonstrations before sampling answers [3, 4, 5]. In NVARC's pipeline, this test-time training (TTT) takes 197 s of a task's 622 s on one L4 GPU under a 12-hour budget, and restarts from the same weights on every task.

We ask whether this adaptation can be amortized: learn once, across many tasks, a map from demonstrations to useful weights, and pay one forward pass per task. Our contributions:

- H, a hypernetwork that reuses the frozen solver as its encoder and emits rank-16 factors for every attention projection.
- Meta-training on task loss, then a first-order MAML stage that trains H's output as a fine-tuning start.
- A paired five-arm evaluation separating H alone, H as TTT's starting factors, and H merged before TTT.

2 Prior work

TTT for ARC. Akyürek et al. [3] showed that TTT on a task's in-context examples raises ARC accuracy up to sixfold over fine-tuned baselines. The ARChitects [4] apply task-specific augmentations in training, depth-first sampling and scoring, with the model's own probabilities selecting answers. NVARC [5], first in ARC Prize 2025, added synthetic puzzles, a Qwen3-4B grid model and per-task LoRA at rank 256.

Hypernetworks and generated adapters. Ha et al. [6] generate one network's weights with another. Text-to-LoRA (T2L) [7] builds a LoRA adapter from a task description in one forward pass, and Doc-to-LoRA (D2L) [8] builds one from a long context. We borrow T2L's head design and D2L's injection of externally supplied factors. HyperDreamBooth [9] composes predicted weights into a diffusion model, then fine-tunes briefly; our merged arm follows that pattern.

Learned starts. MAML [10] and its first-order variants [11] train an initialization for fast fine-tuning; MMAML [12] modulates that initialization per task.

New here: H reads demonstration grids, not text; its encoder is the frozen solver itself; and its output is merged into the base before unchanged TTT.

3 Approach

Base pipeline. NVARC's Qwen3-4B model reads and writes grids in a 16-token vocabulary [5]. Per task, TTT trains a LoRA for one epoch over 128 augmented views of the demonstrations (128 steps); depth-first sampling with augmentation rescoring then picks two attempts per test input. Our development runs train rank 16 on the q, k, v, o projections, which scored no worse than NVARC's rank 256 on our 40-task subset (10.5 against 8.5 tasks).

Encoder. Each demonstration pair passes through the frozen base under 8 augmentations. Mean-pooling the layer-24 hidden states (of 36) over output-grid positions gives $e_i \in \mathbb{R}^{2560}$; averaging over pairs and views gives $\bar{z}$. Embeddings are cached (208,030 in meta-training).

Hypernetwork. Following T2L, H concatenates a code of $\bar{z}$ with layer and module embeddings, runs an MLP trunk, and applies one linear head per module type across all 36 layers. It emits factors A ($16 \times d_{\text{in}}$) and B ($d_{\text{out}} \times 16$) for q, k, v and o, 11.8M values in all, applied as $W' = W + \text{sBA}$ with rsLoRA scale $s = \alpha/\sqrt{r} = 8$ [13]. The B head starts at zero, so the untrained H reproduces the base. H has 134,958,208 parameters.

Objective. An episode holds one task's demonstrations D and a held-out query pair (x, y). With D in context, H minimizes $L = -\log p(y \mid x, D; \theta + \Delta\theta_H(D))$ over output-grid tokens, backpropagating through the frozen base into H.

Training. The 99,674 training episodes come from 51,048 tasks generated by BARC [14] and ReARC [15] or taken from ARC-AGI-2 training; a hash check against the ARC-AGI-2 evaluation set found no overlap. H trained 4,180 steps of 32 episodes on 4×L4, then 80 first-order MAML steps: two inner Adam steps on the demonstrations at rate 4×10^{-4}, then the query loss as the outer loss. Validation uses 286 episodes from 200 held-out tasks.

Three uses. (b) *H alone*: decode with H's adapter, no training. (f) *Factor start*: TTT starts from H's factors. (g) *Merge*: write W + sBA into the bf16 base weights, run standard TTT from a fresh zero-initialized LoRA, and restore W after the task. The linked submission (v19) runs (g) with NVARC's original rank-256 TTT.

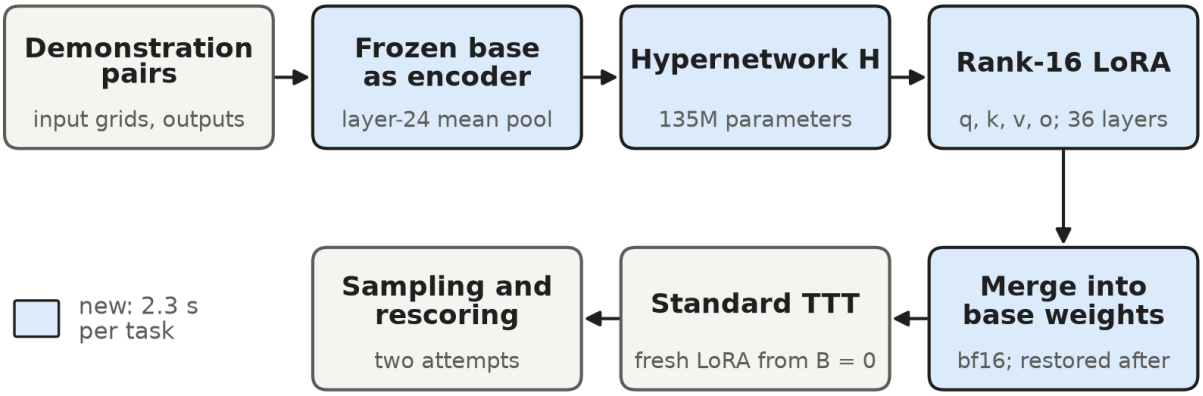


Figure 1. Merge-then-train. The frozen base encodes the demonstrations; H maps the pooled embedding to a rank-16 adapter, which is merged into the base weights; standard TTT then fits a fresh LoRA, and sampling picks two attempts. Shaded steps are new and add 2.3 s per task.

4 Results

Setup. DEV40 is a fixed sample of 40 tasks (55 test outputs) from the public ARC-AGI-2 evaluation set. All five arms ran on every task in one Kaggle 4×L4 session (24 Oct 2026, one seed). A task scores the share of its test outputs solved within two attempts; one task is 2.5 points, and differences under 2 tasks are noise.

Table 1. Five arms on DEV40, paired per task. Seconds are per task on one L4 GPU, all stages included.

Arm	Tasks of 40	Attempt-1 exact of 55	Seconds per task
(a) Base, no adapter	4.0	3	178.3
(b) H alone	5.0	3	160.9
(c) Plain TTT	10.5	8	481.9
(f) TTT from H's factors	9.5	8	456.9
(g) Merge H, then TTT	10.5	10	487.9

Merging ties plain TTT task for task. Arms (g) and (c) solve the same 11 outputs, 10.5 tasks each. (g) gets two more outputs right at attempt 1 (10 against 8), within noise.

The factor start costs fit quality, not tasks. (f) solves 9.5 tasks, −1.0 against (c) (90% paired interval −3 to +1): no difference. But its training loss rises above (c)'s from step 20 and ends higher on 38 of 40 tasks (median ratio 2.2). Merging removes this penalty: (g) reaches (f)'s final loss on all 40 tasks (median step 63) and ends below it on all 40 (median ratio 0.47).

H alone solves 5.0 tasks against the base's 4.0 (one gained, none lost) without training.

Training loss (Figure 2). Step 1 runs at learning rate 0, so its loss measures the start: 0.0916 for (g) against 0.0946 for (c), 3% lower. (g) stays below (c) through step 45; then the curves cross 11 times, ending at 0.0098 against 0.0094. Per task, (g) reaches (c)'s final loss at median step 95 of 128: not sooner.

Cost. H adds 2.3 s per task: encoding 1.9 s, H's forward pass 0.02 s, merging 0.34 s and restoring 0.04 s. Outside decoding, (g) runs within a second of (c).

Leaderboard. v19 scores **[PLACEHOLDER: public score]** public and **[PLACEHOLDER: private score]** private. For context, NVARC's notebook, rerun verbatim, scores 30.97 public; leaderboard differences under 2 points are noise.

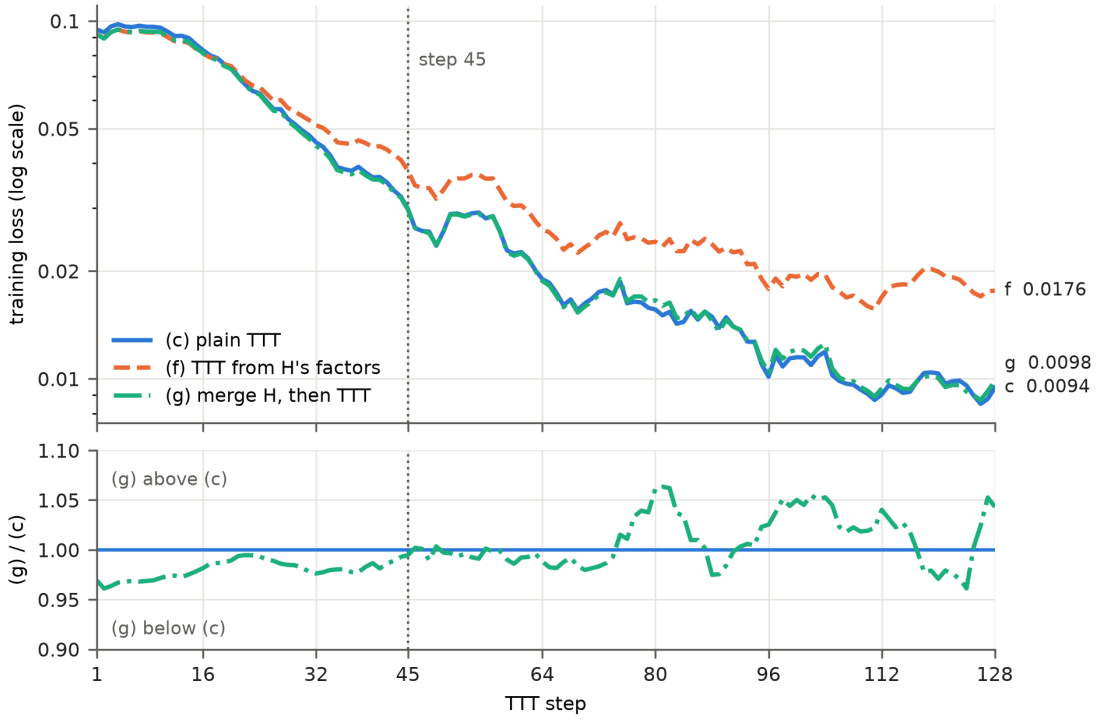


Figure 2. TTT training loss on DEV40 for plain TTT (c), TTT from H's factors (f) and merge-then-TTT (g): mean over 40 tasks, trailing mean over 8 steps, log scale. Lower panel: (g) divided by (c). (g) stays below (c) through step 45 (dotted line), then ties it; (f) rises above (c) from step 20.

5 Conclusion

H is cheap and safe to merge: 2.3 s per task, every base weight restored bit for bit, and a bf16 rounding error of 3% of its delta. Its value shows early in training and is gone by step 128, where it solves exactly what plain TTT solves. Our reading, from one run: a fresh LoRA trained from zero erodes the start's lead within about 45 steps, so the per-task fit, not the start, decides what is solved. Starting from H's factors gives up LoRA's zero-initialized B and fits about half as tightly; merging keeps it. In meta-training, H cut held-out loss on BARC episodes by 17% to 28% but sat above the no-adapter loss on ReARC at 18 of 20 validations; ReARC was 15% of the episode draws, and the base's loss on it was already 2.7 times lower than on BARC.

Limitations. One seed, 40 tasks. DEV40 used rank-16 TTT; v19 uses NVARC's rank-256 TTT, untested with H at this scale. H's held-out loss was flat after step 1,000. Next: (g) against (c) at 32 and 64 training steps, where the early lead could become saved time; episodes closer to ReARC's distribution; a larger rank or a set encoder.

References

[1] Francois Chollet, Mike Knoop, Gregory Kamradt, Bryan Landers and Henry Pinkard. ARC-AGI-2: A New Challenge for Frontier AI Reasoning Systems. arXiv:2505.11831, 2026.

[2] Edward J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685, 2021.

[3] Ekin Akyürek et al. The Surprising Effectiveness of Test-Time Training for Few-Shot Learning. arXiv:2411.07279, 2025.

[4] Daniel Franzen, Jan Disselhoff and David Hartmann. Product of Experts with LLMs: Boosting Performance on ARC Is a Matter of Perspective. arXiv:2505.07859, 2025.

[5] Ivan Sorokin and Jean-François Puget. NVARC Solution to ARC-AGI-2 2025. https://github.com/1ytic/NVARC, 2025. Writeup nvarc_2025.pdf and code, NVIDIA.

[6] David Ha, Andrew Dai and Quoc V. Le. HyperNetworks. arXiv:1609.09106, 2016.

[7] Rujikorn Charakorn, Edoardo Cetin, Yujin Tang and Robert Tjarko Lange. Text-to-LoRA: Instant Transformer Adaption. arXiv:2506.06105, 2025.

[8] Rujikorn Charakorn, Edoardo Cetin, Shinnosuke Uesaka and Robert Tjarko Lange. Doc-to-LoRA: Learning to Instantly Internalize Contexts. arXiv:2602.15902, 2026.

[9] Nataniel Ruiz et al. HyperDreamBooth: HyperNetworks for Fast Personalization of Text-to-Image Models. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. doi:10.1109/cvpr52733.2024.00624

[10] Chelsea Finn, Pieter Abbeel and Sergey Levine. Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks. arXiv:1703.03400, 2017.

[11] Alex Nichol, Joshua Achiam and John Schulman. On First-Order Meta-Learning Algorithms. arXiv:1803.02999, 2018.

[12] Risto Vuorio, Shao-Hua Sun, Hexiang Hu and Joseph J. Lim. Multimodal Model-Agnostic Meta-Learning via Task-Aware Modulation. arXiv:1910.13616, 2019.

[13] Damjan Kalajdzievski. A Rank Stabilization Scaling Factor for Fine-Tuning with LoRA. arXiv:2312.03732, 2023.

[14] Wen-Ding Li et al. Combining Induction and Transduction for Abstract Reasoning. arXiv:2411.02272, 2024.

[15] Michael Hodel. Addressing the Abstraction and Reasoning Corpus via Procedural Example Generation. arXiv:2404.07353, 2024.